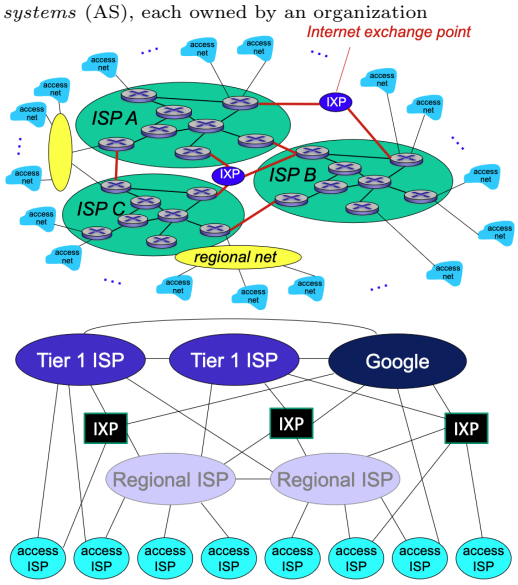
Basics

• **Network**: mesh of interconnected routers
• **Circuit switching**: end-to-end circuit. Shares link bandwidth. Commonly used in traditional telephone networks.
• **Packet switching**: breaks messages into smaller packets, transmits them onto links. Packets passed from one router to the next. *Store-and-forward*: entire packet must arrive before it is retransmitted. Routers use algorithms and addressing to determine where to route packets.
• **Internet structure**: Hosts connect to Internet via access ISPs (Internet Service Providers). Access ISPs in turn must be interconnected. The Internet is a "network-of-networks", organized into *autonomous systems* (AS), each owned by an organization



• **Organizations**: *Network Information Centre (NIC)*: Administers IP address & Internet Naming. *Internet Society (ISOC)*: Provides leadership in Internet related standards, education, and policy around the world. *Internet Architecture Board (IAB)*: Authority to issue and update technical standards regarding Internet protocols. *Internet Engineering Task Force (IETF)*: Protocol engineering, development and standardization arm of the IAB. Internet standards are published as RFCs (Request For Comments)
• **Transmission delay**: time needed to push all bits of a packet onto a link. $d_{trans} = L/R$, where L = packet length (in bits), R = transmission rate/bandwidth (in bits/sec)
• **Propagation delay**: time taken for single bit to travel through physical medium from sender to receiver. $d_{prop} = d/s$, d = length of physical link, s = propagation speed in medium (typically speed of light $\approx 2 \cdot 10^8$ m/s)
• **Processing delay**: check bit errors, determine output link. Typically < milliseconds
• **Queueing delay**: Time waiting in queue for retransmission. Depends on congestion level of router. Queue/buffer of a router has finite capacity, packets arriving to full queue will be dropped
• **Packet delay**: Delay $d = d_{prop} + d_{trans} + d_{proc} + d_{queue}$ (propagation, transmission, processing, queueing
• **Throughput**: how many bits can be transmitted per unit time. Measured for end-to-end communication. Link capacity (bandwidth) is meant for a specific link

Application Layer

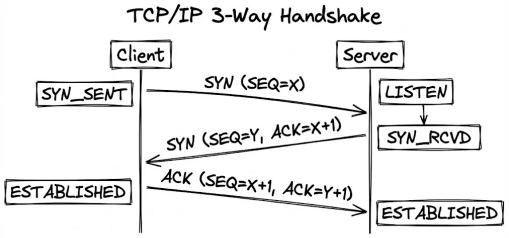
• **Types of architecture**: client-server (server waits for incoming request, client initiates), peer-to-peer (direct end-to-end communication, no central server, self scalable)
• **Transport requirements**: Data integrity: 100% rdt (file transfer, transactions) vs tolerate loss (e.g. streaming). Throughput: minimum bandwidth vs use whatever is available. Timing: low delay (e.g. online games). Security: encryption, authentication
• **HTTP (Hypertext Transfer Protocol)**: client-server, request-response model. Stateless (server does not retain session data, unless cookies/tokens/session ID is used.
• HTTP uses TCP: 1) Client initiates TCP connection. 2) server accepts TCP connection. 3) HTTP messages are exchanged. 4) connection closed
• Web page = base HTML file + referenced objects
• **RTT**: time for packet to travel from client to server and back
• **Non-Persistent** (HTTP/1.0): one object per connection. Response time per file = RTT (connect) + (RTT + file transmission time)
• **Persistent** (HTTP/1.1): multiple object sent over same connection. With pipelining: Little as one RTT for all referenced objects. Response time (for all objects) = RTT (connect) + (RTT + sum of file sizes in bits / transmission rate)
• **Request Format**: `(Request line)method URL version (CRLF) (header lines...) (CRLF) (empty line) (CRLF) (data/payload)`
• **Request methods**: HTTP/1.0 - `GET/POST/HEAD` (leave object out), HTTP/1.1 - `GET/POST/HEAD/PUT/DELETE`
• **Response message format**: Same as request format. 1) status line (version + status code), 2) header lines, 3) blank line, 4) data. Status code: 100-199 (information), 200-299 (success), 300-399 (redirect), 400-499 (client error), 500-599 (server error)
• **Cookies**: allows HTTP request to carry "state"
• **DNS (Domain Name System)**: map from hostname (`www.example.org`) to IP
• **RR (Resource Record)**: the mapping itself. Stored in distributed database according to hierarchy. Format: `Name TTL Class Type RDATA / (name, value, type, TTL)` (course convention). CNAME record: alias (e.g. `blog.example.com`) → canonical hostname (e.g. `example.com`). A Record: hostname → IPv4. NS Record: hostname → nameserver. NS record cannot point to CNAME record. AAAA: hostname → IPv6. MX: domain → mail server name
• **Nameserver**: stores all DNS records for a domain, like A, MX, CNAME etc. Most domains use multiple nameservers for reliability. Typically one main primary NS and several secondary, which store exact copies of the primary. When NS record updated, it takes awhile for the change to be replicated
• **Hierarchy**: there are 13 *root name servers* A-M, each operated by different org. Root server returns list of *Top-Level Domain* (TLD) servers e.g. `.com/.org/.net/.sg`. TLD servers return the *authoritative name server* for the particular website
• **Local DNS server**: i.e. default name server. Each ISP has one. Not part of the hierarchy
• **Authoritative Name Server**: contains the actual DNS records
• **DNS Caching**: If local DNS server has cached query, 1 RTT. Otherwise walk the DNS hierarchy. Cached entries have TTL. If host changes IP address, may not be known Internet-wide until all TTLs expire
• **Iterative Query**: client→local, local↔root, local↔TLD, local↔authoritative, local←client (4 RTT)
• **Recursive Query**: client → root → TLD → authoritative (then back the same way)
• **Port Numbers**: 16-bit integer. 0-1023 are reserved (well-known port numbers) assigned by IANA. 21: FTP, 22: SSH, 25: SMTP, 53: DNS, 80: HTTP, 443: HTTPS
• **TCP socket**: When contacted by client, server creates new socket. Server uses (client IP + port #) to distinguish clients. When client creates its socket, client TCP establishes connection to server TCP
• **UDP socket**: Server uses one socket to serve all clients. No connection is established before sending data. Sender explicitly attaches destination IP address and port # to each packet. Transmitted data may be lost or received out-of-order.

Transport Layer

• Deals with *logical* communication between *processes*. Sends *segments*
• Mux: Gather data from different sockets on a host. Demux: Send data to correct socket on a host
• **UDP (User Datagram Protocol)**: process-to-process delivery and error checking. Unreliable: segments can arrive out of order or not at all. Unregulated sending rate (no congestion control). Small header, no connection overhead
• **UDP Header** (always 8 bytes): src port (16), dest port (16), length (16), checksum (16). Checksum: 16-bit sum over header, payload, with carry bit wrap around, then take 1's compliment. Checksum is 0 during calculation
• **Unconnected UDP**: `serverSocket = socket(AF_INET, SOCK_DGRAM); serverSocket.bind(('', serverPort));`

```
msg, clientAddr = serverSocket.recvfrom(2048);
serverSocket.sendto(reply, clientAddr);
serverSocket.close();
clientSocket.sendto(msg.encode(), (serverName, serverPort))
msg, serverAddr = clientSocket.recvfrom(2048)
msg.decode() # byte to string
```

• DNS server/DHCP/game server use unconnected UDP. Suitable for one-to-many. If connected, socket is locked to a single peer and drops everyone else
• **Connected UDP**: `clientSocket.connect((serverName, serverPort)); clientSocket.send(msg.encode()); responseStr = clientSocket.recv(2048).decode()`
• **TCP (Transmission Control Protocol)**: has Reliable Data Transfer (*rdt*), congestion control (throttle sender when network is overloaded), flow control (sender won't overwhelm receiver). Three-way handshake to establish connection



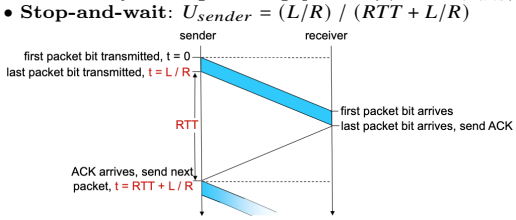
• TCP socket (for (de)mux)) identified by 4-tuple (src ip, src port, dest ip, dest port)
• **TCP client/server**: `clientSocket = socket(AF_INET, SOCK_STREAM); clientSocket.connect((serverName, serverPort)); # no need to include server name/port; clientSocket.send(message.encode()); msg = clientSocket.rcv(2048)`

```
# tcp server
socket/bind/listen/accept/send/rcv
```

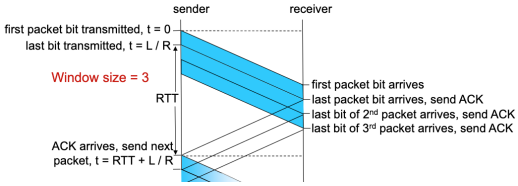
• TCP Header: 20 bytes, up to 60 (variable)

Source Port (16)									Destination Port(16)
Sequence Number(32 bits)									
Acknowledgement Number(32 bits)									
Header Length (4 bits)	6 Bits Reserved	URG (1 bit)	ACK (1 bit)	PSH (1 bit)	RST (1 bit)	SYN (1 bit)	FIN (1 bit)	Window Size(16 bits)	
Checksum(16 bits)									Urgent Pointer(16 bits)
Options/Optional data(0-40 bytes)									

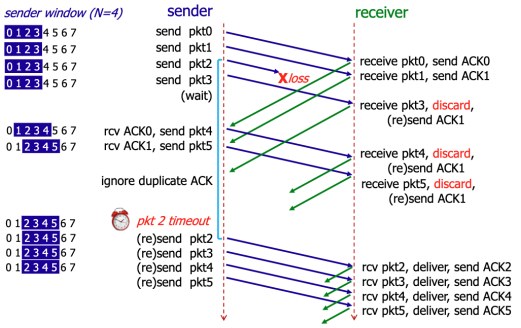
• SYN, ACK bits used for 3 way handshake. FIN for 4 way teardown. RST (abort immediately). PSH (push data now, don't buffer). URG (urgent pointer points to byte after urgent data, seqnum + offset), rarely used
• Seqnum: byte number of first byte in a data segment. ACK number: seq# of next byte expected by receiver. TCP ACKs until first missing byte (cumulative ACK)
• **Checksum**: calculation same as UDP
• rdt 1.0: Assume channel is perfectly reliable
• rdt 2.0: Bit error, detect using checksum. Server sends NACK. Stop-and-wait: Sender sends one packet at a time, then waits for response. Flaw: server may send corrupted ACK/NACK, client may retransmit correctly received packet
• rdt 2.1: rdt 2.0 + seq num. Receiver discards duplicate packets
• rdt 2.2: send ACK only. Include seq num of **last** packet received OK
• rdt 3.0: Assumes underlying channel may flip bits, lose packets, have long delay, but **won't reorder packets**. To detect packet loss: sender waits for ACK, retransmits if not received by timeout (regardless whether due to packet loss or delay). Receiver discard duplicate using seqnum, and specify seqnum in ACK. If no ACK or receive ACK but for wrong packet → packet loss
• **Utilization**: $d_{trans}/(RTT + d_{trans})$, fraction of time sender is busy sending. Throughput = $L/(RTT + d_{trans})$
• **Stop-and-wait**: $U_{sender} = (L/R) \ / \ (RTT + L/R)$

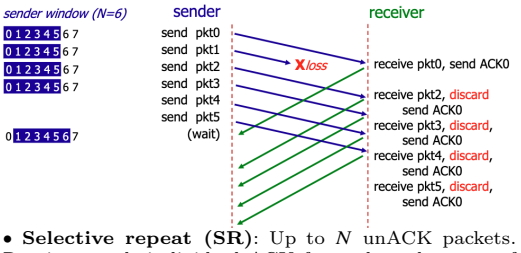


• **Pipelining**: $U_{sender} = (k \cdot L/R) \ / \ (RTT + L/R)$ where k is window size

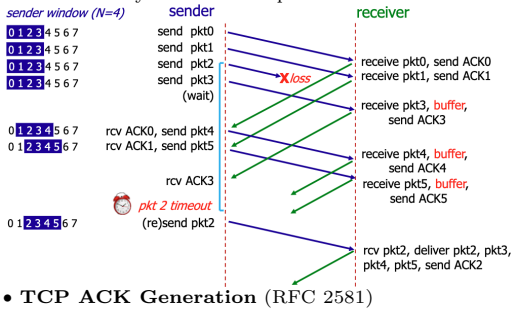


• **Go-Back-N (GBN)**: Up to N unACK packets in pipeline. Receiver only sends cumulative ACK, out of order packets discarded. Sender sets timer for oldest unACKed packet. When timer expires, retransmit packet n and subsequent packets in window





• **Selective repeat (SR)**: Up to N unACK packets. Receiver sends individual ACK for each packet, out of order packets ACKed and buffered. Sender maintains timer for each unACK packet. When timer expire, retransmit only that unACK packet



• **TCP ACK Generation** (RFC 2581)

Event at TCP receiver	TCP receiver action	
Arrival of in-order segment with expected seq #. All data up to expected seq # already ACKed	Delayed ACK: wait up to 500ms for next segment. If no next segment, send ACK	
Arrival of in-order segment with expected seq #. One other segment has ACK pending	Immediately send single cumulative ACK, ACKing both in-order segments	
Arrival of out-of-order segment higher-than-expect seq. # (gap detected)	Immediately send *duplicate ACK*, indicating seq. # of next expected byte	
Arrival of segment that partially or completely fills gap	Immediately send ACK, provided that segment starts at lower end of gap	

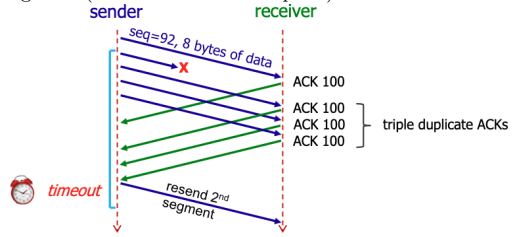
• **TCP Timeout value**: too short timeout: premature timeout and unnecessary retransmissions. Too long timeout: slow reaction to segment loss. Timeout interval must be longer than RTT, but RTT varies. Thus TCP computes (and keeps updating) timeout interval based on *estimated RTT*

• $EstimatedRTT = (1 - \alpha) \cdot EstimatedRTT + \alpha \cdot SampleRTT$. Typical α 0.125

• Deviation $DevRTT = (1 - \beta) \cdot DevRTT + \beta \cdot |SampleRTT - EstimatedRTT|$. Typical β 0.25

• TimeoutInterval = EstimatedRTT + 4*DevRTT (safety margin)

• **TCP Fast retransmission**: Timeout period is often relatively long: long delay before resending lost packet. Fast retransmission: If sender receives 4 ACKs for the same segment, it supposes that segment is lost → resend segment (even before timer expires)



• **4 way teardown**: client FIN seq u, server ACK seq u+1, server FIN seq t, client ACK seq t+1

Network Layer

• Provides *logical* communication between *hosts*. Sends *IP datagrams*

• **Internet Protocol (IP)**: *Best effort* delivery (no guarantees) - unreliable.

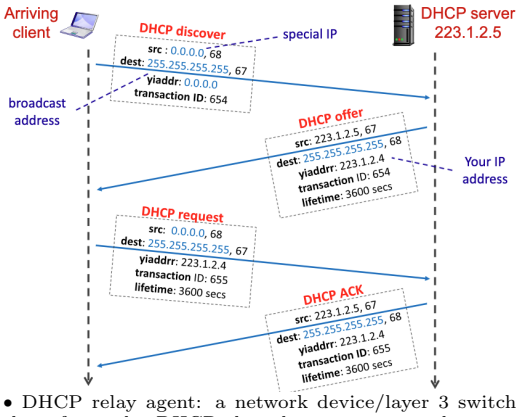
• **IP address**: 32 bit integer used to identify a host/router, in binary or dotted decimal. Comprises two parts: network (subnet) prefix + host ID

• **Subnet**: A network of "directly" interconnected hosts. Hosts in the same subnet have the same network prefix, and can physically reach each other without router. They connect to outside through a router

• **IP Address Allocation**: Organization buy from registry or rent from ISP's address space. ISP from *ICANN (Internet Corporation for Assigned Names and Numbers)* - Allocates addresses, Manages DNS, Assigns domain names, resolves disputes

• **CIDR (Classless Inter-domain Routing)**: IP address assignment strategy. Format: `a.b.c.d/x`, x is number of subnet prefix bits

• **DHCP (Dynamic Host Configuration Protocol)**: App layer protocol to assign IP address and configuration to devices when they connect to a network. DORA: Discover (broadcast msg to find DHCP server), Offer (server replies with IP and settings), Request (ask to be assigned), Acknowledgment. DHCP server: dedicated computer managing pool of IP addresses. Lease time: temporary period allowed to use IP. DHCP uses UDP



• DHCP relay agent: a network device/layer 3 switch that forwards DHCP broadcast messages between clients and server located on different subnet. Packages broadcast msg into unicast msg, adds its own IP to `giaddr` (gateway IP addr) field so server knows which network pool to assign from. Eliminates need to install DHCP server on every network. Allows single DHCP server to manage multiple subnets. `giaddr` is 0.0.0.0 if client and server in same subnet

• DHCP runs on UDP. Server port: 67, client port 68

• **Special IP addresses**: `10.0.0.0/8`, `172.168.0.0/12`, `192.168.0.0/16` (Private addresses, can be used without any coordination with IANA or an Internet registry). Loopback: `127.0.0.0/8`, tests network software on the local device; `127.0.0.1` is standard localhost address. Broadcast (`255.255.255.255`): Sends packets to all devices simultaneously on a local subnet. Unspecified (`0.0.0.0`): Used as a placeholder or default route during network initialization

• **Subnet Mask**: determine which subnet an IP address belongs to. Zero out host bits. # of usable Hosts = $2^n - 2$, n is the number of host bits (-2 for network (first, all host bits 0) and broadcast address (last, all host bits 1)). Special rules apply for /31 (1 usable host, used for point-to-point links) and /32 (0 usable hosts, represents a single host)

• To create subnets of equal size, borrow from host bits smallest k s.t. $2^k \geq$ subnet size

• **Longest Prefix Match**: match inbound packet with *forwarding table* entry with longest prefix match

• **Routing**: The Internet is a "network-of-networks". A hierarchy of *Autonomous Systems* (AS) e.g. ISPs each owns routers and links. Routing is done hierarchically. **Intra-AS**: Find good path between two routers within an AS. Commonly used protocols: *RIP, OSPF*. Performance over policy. **Inter-AS**: *BGP* (de-facto standard), dominated by policy

• **Routing algorithm types**: *link-state*: all routers have complete knowledge of topology and link cost, periodically broadcast cost - use Dijkstra's. *distance vector*: only know neighbours costs, exchange update with neighbours

• **Bellman Ford**: $d_x(y) = min_{v \in N(x)}\{c(x,v) + d_v(y)\}$

• **Count to Infinity problem for DV**: routers endlessly increment their cost metric to broken destination. Routers only share distance information with their neighbours. E.g. A-B-C, B-C link breaks, but B thinks A can still reach C, updates distance to C via A. Solutions: Split horizon, route poisoning (advertise infinite distance/unreachable), Hold-down timers (temporary refuse updates for suspect route)

• **RIP (Routing Information Protocol)**: implements the DV algorithm. Uses *hop count* as the cost metric (insensitive to network congestion). Exchange routing table every 30 seconds over UDP port 520. Self-repair: if no update from a neighbour router for 3 minutes, assume neighbour has failed

• **NAT (Network Address Translation)**: all datagrams leaving local network have the same source NAT IP address. Within local network, hosts use private IP addresses. NAT routers must: 1) Replace (src IP, port) of every outgoing datagram to (NAT IP, new port #), 2) Remember mapping in translation table, 3) replace incoming datagrams using mapping. Benefits: no need rent public IP range from ISP, just one public IP for NAT router. Hosts use private IP, can change freely, not addressable by outside (more secure). Can change ISP without affecting hosts. NAT may change src IP, dest IP, TTL, checksum of inbound/outbound IP datagrams

• **IP Header Format** (20 bytes, up to 60)

Bit	+0..7		+8..15		+16..23	+24..31
0	Version	Header Length	DSCP	ECN	Total Length	
32	Identification				Flags	Fragment Offset
64	Time To Live		Protocol		Header Checksum	
96	Source IP Address					
128	Destination IP Address					
160	Options (if present)					
...	*Payload*					

• **Fragmentation**: Identifier: original id packet, used to rebuild. Flags(3): reserved (always 0), DF (don't fragment, drop if can't forward (size > link's MTU, send ICMP "Fragmentation Needed"), MF (must fragment, 1 for more to follow, 0 for last fragment)

• **# of fragments needed**: Fragment Offset is **measured in 8-byte units**, so every fragment's payload (except the last) **must be a multiple of 8**. 1) Compute actual payload size = IP datagram size - IP header. 2) Compute max payload per fragment = MTU - IP header, then round down to multiple of 8. 3) # Fragments = ceil(payload size/max payload per fragment)

• **MTU (Maximum Transmission Unit)**: max size that can be sent over link in a single frame. It includes IP header size. Max payload per fragment = MTU - IP header size. (MTU is layer 2/3, MSS (Maximum Segment Size) is layer 4 TCP, payload size only. MSS = MTU - IP header - TCP header)

• A device can have multiple IPs: `lo` (loopback 127.0.0.1), `eth0` (ethernet), `wlan0` (wifi). `0.0.0.0` is a *wildcard address*, socket bind to `0.0.0.0:<port>` means accept all traffic for any IP on that port. Wildcard can be used as fallback. Can have multiple sockets on a port. More specific `IP:port` beats wildcard. `SO_REUSEADDR` / `SO_REUSEPORT` needed for multiple sockets on same port, otherwise bind fails with "address already in use"

• **Upper-Layer Protocol Number**: an 8-bit value in IP header that tells receiving system which upper-layer protocol should handle the data (1=ICMP, 6=TCP, 17=UDP, 41=IPv6, 89=OSFP)

• **Explicit Congestion Notification (ECN)**: two bits in IP header. Allows routers to notify end devices of network congestion before packets are actually dropped. 00=sender not ECN capable, router drops packet on overflow. 01/10=ECN-capable Transport (ECT). 11=Congestion Experienced (CE). Receiver sees CE flag, sends ECE flag in response. Sender sends CWR (Congestion Window Reduced) flag, throttles. Default=00, not set until explicitly negotiated

• **ICMP (Internet Control Message Protocol)**: Used by hosts and routers to communicate network-layer information to each other. Mostly for error reporting e.g. destination unreachable, but can be used for ping/echo request. Architecturally above IP (ICMP messages are carried as IP payload)

• Format: type(8), code(8), checksum(16), extended header(32), payload (Total 8 bytes). Error msg contains original IP **header** plus **first 8 bytes** of payload (TCP/UDP stores src/dest port here) for sender to identify process

• Type8Code0=Echo request. Type0Code0=Echo reply. Type4Code0=Source quench (congestion control, force host to reduce transmission rate). Type 3 = destination error, 11 = TTL expire, 12 = IP bad

• *traceroute* is implemented using ICMP by sending UDP packets with increasing TTL until destination replies. Receives ICMP error message with TTL expiry, logs each intermediate router's and RTT